RELATÓRIO TÉCNICO | ASSEMBLEIA GERAL

GASPAR

Arquitetura, Tecnologias e Mecanismos de Decisão

1. Visão geral da arquitetura

O GASPAR é uma aplicação corporativa de gestão por processos de negócio (*BPM*) construída sobre uma **arquitetura orientada a metadados**: processos, formulários, situações, regras de encaminhamento e permissões são armazenados como dados e configurados pelo Administrador por interface, sem alteração do código-fonte e sem novas migrações estruturais de banco de dados.

Camada	Tecnologias e componentes	Responsabilidade
Apresentação	Filament v3, Livewire 3, Alpine.js, Tailwind CSS	Painel administrativo reativo, visão Kanban ou lista, formulários dinâmicos e modo escuro.
Aplicação e regras	Laravel 12 (PHP 8.3), Policies, Middlewares	Autorização, regras de negócio, serviços de transição e emissão de PDF.
Motor de workflow (BPM)	DemandTransitionService, TransitionDecisionEvaluator	Orquestração de situações, gateways de decisão (DMN) e cálculo de SLA.
Persistência	MySQL 8.0+ / MariaDB, Eloquent ORM, UUIDv7	Dados e metadados, identificadores cronológicos e auditoria completa.

- **UUIDv7** em todas as entidades: ordenação cronológica natural e menor fragmentação de índices.
- **Banco de dados:** MySQL / MariaDB, com compatibilidade com SQLite em memória para testes automatizados.
- **Relatórios:** DomPDF converte templates estruturados em PDF formal.

2. Digitalização de um processo pelo Administrador

A digitalização de um processo é feita integralmente pelo Administrador na interface, em seis etapas. Cada etapa corresponde a um conjunto de metadados que o motor do sistema interpreta em tempo de execução:

Etapa	O que o Administrador define	Elemento técnico
1. Processo	Cria o tipo de processo (ex.: Contratação de Serviços, Compras, Viagens).	custom_entities
2. Campos	Monta o formulário: tipo (texto, número, data, seleção, checkbox, radio, arquivo), obrigatoriedade, opções, valor padrão e ordem.	custom_fields e pivô custom_entity_custom_field
3. Situações	Define as etapas do fluxo (ex.: Nova, Licitando, Aprovando, Adquirindo, Quitando, Encerrada, Avaliada).	process_statuses
4. Transições e regras	Define o caminho entre as situações: direto ou condicional, com condições por ordem de prioridade, situação padrão e permissão de retorno.	StatusTransition, transition_rules
5. Papéis e permissões	Define, por papel e por situação, se cada campo é obrigatório, opcional, somente leitura ou oculto, e quem atua em cada etapa.	FieldPermissionResolver
6. Prazo (SLA)	Define as horas padrão do processo.	sla_hours

Concluída a configuração, o processo fica disponível para abertura de demandas. Alterações posteriores de formulário, fluxo ou regras não exigem desenvolvimento, nem implantação de nova versão.

3. Engine dinâmica de metadados

Para suportar vários tipos de processo sem criar tabelas físicas para cada um, utiliza-se uma derivação do padrão **EAV** (*Entity-Attribute-Value*): a definição dos campos é um dado, e as respostas de cada demanda são gravadas de forma genérica.

```
Processo (custom_entities)
   -> Campos associados (custom_entity_custom_field)
      -> Definição do campo (custom_fields)
         tipos: texto, número, data, seleção, checkbox, radio, arquivo
         regras: obrigatório, opções, valor padrão, ordem
      -> Respostas da demanda (demand_field_values)
         demanda + campo + valor (texto ou JSON)
```

- **Serialização:** valores escalares são gravados diretamente; listas e seleções múltiplas, em JSON.
- **Formulário dinâmico:** `buildDynamicFieldSchema()` lê os campos do processo e monta o formulário em tempo de execução.
- **Permissão por campo:** `FieldPermissionResolver` avalia a situação da demanda e o papel do usuário para definir o modo de cada campo.

4. Motor de transições e gateways de decisão

O `DemandTransitionService`, apoiado pelo `TransitionDecisionEvaluator`, controla o encaminhamento entre situações. Há dois tipos de transição:

- **Direta:** a demanda segue para uma situação previamente determinada.
- **Condicional (gateway DMN):** o destino depende de regras cadastradas. As condições são avaliadas por ordem de prioridade; a primeira satisfeita define o destino e, se nenhuma for atendida, aplica-se a situação padrão (*fallback*).

Exemplo configurado: conclusão da etapa de licitação

Ordem	Condição avaliada	Situação de destino
1	Preço acima de Y	Aprovando Alto (Diretoria)
2	Preço acima de X	Aprovando (Gestão)
Padrão	Nenhuma das anteriores	Adquirindo (compra direta)

Operadores disponíveis na configuração de regras

Tipo de campo	Operadores
Numéricos e monetários	Igual, diferente, maior, maior ou igual, menor, menor ou igual e entre. Normaliza o formato brasileiro (R$ 10.000,00 passa a 10000.00).
Datas	Igual, antes, depois e entre.
Textos e seleções	Igual, diferente, contém, está na lista, vazio e preenchido.

5. Ciclo de vida da demanda e travas de integridade

O estado de cada demanda é regulado por dois eixos complementares:

- **Eixo operacional:** Rascunho, Em Atendimento, Encerrada, Avaliada ou Cancelada.
- **Eixo de fluxo de trabalho:** situação do processo, definida pelo Administrador (ex.: Nova, Licitando, Aprovando).

Trava	Funcionamento
Subdemandas abertas	Impede o encerramento ou cancelamento da demanda principal enquanto houver subdemandas em rascunho ou ativas.
Ação de retorno	Permitida somente se a transição de origem admitir retorno e o operador for o responsável atual da demanda ou administrador.
Imutabilidade do processo	O tipo de processo é bloqueado contra alteração após a criação da demanda, preservando a integridade entre campos e situação.
Situações nativas	Situações do sistema (Nova, Encerrada e Cancelada) são protegidas contra exclusão acidental ou em massa.

6. Prazos (SLA) e monitoramento automatizado

- **Cálculo automático:** na criação, se o processo possui horas de SLA, o prazo limite é calculado automaticamente.
- **Início oficial:** na primeira transição para fora da situação inicial, registra-se a data de início e o prazo é recalculado a partir dela.
- **Verificação periódica:** o comando `app:check-demand-deadlines` roda por agendamento (cron) ou por rota administrativa protegida.
- **Alertas:** preventivo um dia antes, de urgência no dia do vencimento e diário enquanto vencida e aberta, limitado a um por dia por demanda.

7. Segurança, permissões e auditoria

Mecanismo	Descrição
Rotas administrativas	Rotas sensíveis (atualização, verificação de prazos, teste de e-mail e desinstalação) exigem usuário autenticado com perfil Administrador; demais acessos recebem resposta 403.
Auditoria estruturada	Alterações críticas e transições registram operador, entidade, valores anteriores e novos, endereço IP e navegador.
Resiliência do e-mail	Falhas momentâneas do servidor SMTP são registradas em log, sem interromper a transação nem exibir erro ao usuário.

8. Painel do demandante

O painel mantém em destaque as demandas encerradas ainda não avaliadas pelo solicitante, com indicação visual e botão de ação rápida. Enviada a nota, a demanda passa ao estado Avaliada e é arquivada das pendências imediatas.

9. Qualidade e testes automatizados

A suíte automatizada (PHPUnit / Pest) possui **48 testes aprovados e 173 asserções**:

- **Unidade:** resolução de permissões por papel e situação; operadores do avaliador de regras e *fallback*.
- **Integração:** gateways, validação de campos obrigatórios e bloqueio por subdemandas.
- **Ponta a ponta (contratação):** roteamento por faixas de preço, retorno por reprovação, notificações, bloqueio de rotas administrativas, fluxo de avaliação, SLA e emissão do PDF.

10. Conclusão técnica

O GASPAR apresenta arquitetura extensível e alinhada aos padrões atuais do ecossistema Laravel e de BPM. A configuração por metadados permite ao Administrador digitalizar e evoluir processos de forma autônoma, enquanto a separação entre motor de regras, serviços de transição e interface reativa favorece a manutenibilidade, o desempenho e a confiabilidade.